

TI — Load_Revenue_FromCSV

Paramètres du processus

NOM	TYPE	VALEUR PAR DÉFAUT	PROMPT
pFilePath	String	revenue_data.csv	Chemin complet du fichier CSV
pVersion	String	Forecast	Version cible (ex: Actual, Forecast, Version 1)
pDelimiter	String	,	Séparateur CSV (virgule par défaut)
pClearBeforeLoad	String	N	Vider la version avant chargement ? (Y/N)

Configuration datasource : Type = ASCII · Delimiter variable = pDelimiter · Header rows = 1 · Quote character = " Variables mappées : V1=Organization · V2=Channel · V3=Product · V4=Month · V5=Year · V6=Version · V7=UnitsSold · V8=UnitPrice · V9=UnitCost

Prolog

PROLOG

Exécuté **une seule fois avant** la lecture du fichier. Validation des paramètres, initialisation des compteurs, clear optionnel.

```
#####
# Load_Revenue_FromCSV — PROLOG
# Objectif : valider les paramètres, init compteurs, clear opt.
# Cube cible : Revenue (organization, Channel, product,
#             Month, Year, Version, Revenue)
#####

#---- 1. Validation des paramètres obligatoires ----
IF( TRIM(pFilePath) @= ' ' );
    ProcessError();
    LogOutput( 'ERROR', 'Load_Revenue_FromCSV: pFilePath est vide. Arrêt du processus.' );
    ProcessBreak();
ENDIF;

IF( TRIM(pVersion) @= ' ' );
    ProcessError();
    LogOutput( 'ERROR', 'Load_Revenue_FromCSV: pVersion est vide. Arrêt du processus.' );
    ProcessBreak();
ENDIF;

#---- 2. Vérification que la version cible est une version de saisie ----
# Bloquer les versions calculées (Variance, Rate Variance, etc.)
IF( pVersion @= 'Variance'
    % pVersion @= 'Variance%'
    % pVersion @= 'Rate Variance'
    % pVersion @= 'Volume Variance'
    % pVersion @= 'Ccy Exchange Variance'
    % pVersion @= 'Performance Variance'
    % pVersion @= 'Target vs Prior Year Actual'
    % pVersion @= 'Target vs Prior Year Actual %' );
    ProcessError();
    LogOutput( 'ERROR', 'Load_Revenue_FromCSV: Version "' | pVersion | '" est une version calculée — écr
```

```

ENDIF;

#---- 3. Vérification que la version existe dans la dimension ----
IF( DIMNM( 'Version', DIMIX( 'Version', pVersion ) ) @= ' ' );
    ProcessError();
    LogOutput( 'ERROR', 'Load_Revenue_FromCSV: Version "' | pVersion | '" introuvable dans la dimension
    ProcessBreak();
ENDIF;

#---- 4. Clear optionnel avant chargement ----
IF( pClearBeforeLoad @= 'Y' );
    LogOutput( 'INFO', 'Load_Revenue_FromCSV: Clear de la version "' | pVersion | '" avant chargement.'
    CubeClearData( 'Revenue' );
    ## Note : en prod, remplacer par un CellPutN ciblé sur la version
    ## ou utiliser une vue filtrée pour éviter de vider toutes les versions
ENDIF;

#---- 5. Initialisation des compteurs ----
nLignesChargees = 0;
nLignesIgnorees = 0;
nLignesErreur   = 0;

LogOutput( 'INFO', 'Load_Revenue_FromCSV: Démarrage - fichier=' | pFilePath | ' version=' | pVersion )

```

Data

DATA

Exécuté **pour chaque ligne** du fichier CSV (hors entête). Validation ligne par ligne, mapping vers les membres réels du cube
Revenue, écriture des 3 mesures.

```

#####
# Load_Revenue_FromCSV - DATA
# Colonnes CSV : V1=Organization, V2=Channel, V3=Product,
#                 V4=Month, V5=Year, V6=Version,
#                 V7=UnitsSold, V8=UnitPrice, V9=UnitCost
#####

#---- 1. Lire les valeurs de la ligne courante ----
sOrg      = TRIM( V1 );
sChannel  = TRIM( V2 );
sProduct  = TRIM( V3 );
sMonth    = TRIM( V4 );
sYear     = TRIM( V5 );
sVersion  = TRIM( V6 );
nUnitsSold = StringToNumber( V7 );
nUnitPrice = StringToNumber( V8 );
nUnitCost  = StringToNumber( V9 );

#---- 2. Ignorer les lignes vides ou de type entête résiduel ----
IF( sOrg @= ' ' % sOrg @= 'Organization' );
    nLignesIgnorees = nLignesIgnorees + 1;
    ItemSkip();
ENDIF;

#---- 3. Forcer la version depuis le paramètre (le CSV peut indiquer
#         une version différente - pVersion fait toujours foi) ----
sVersion = pVersion;

```

```

#---- 4. Validation : Organisation ----
IF( DIMNM( 'organization', DIMIX( 'organization', sOrg ) ) @= ' ' );
    LogOutput( 'WARN', 'Ligne ignorée - Organization inconnue: [' | sOrg | ']' );
    nLignesIgnorees = nLignesIgnorees + 1;
    ItemSkip();
ENDIF;

#---- 5. Validation : Channel (membres réels : 10, 20, 30) ----
IF( DIMNM( 'Channel', DIMIX( 'Channel', sChannel ) ) @= ' ' );
    LogOutput( 'WARN', 'Ligne ignorée - Channel inconnu: [' | sChannel | ']' (valeurs valides: 10, 20, 30) );
    nLignesIgnorees = nLignesIgnorees + 1;
    ItemSkip();
ENDIF;

#---- 6. Validation : Produit ----
IF( DIMNM( 'product', DIMIX( 'product', sProduct ) ) @= ' ' );
    LogOutput( 'WARN', 'Ligne ignorée - Product inconnu: [' | sProduct | ']' );
    nLignesIgnorees = nLignesIgnorees + 1;
    ItemSkip();
ENDIF;

#---- 7. Validation : Month (membres réels : Jan, Feb...Dec) ----
IF( DIMNM( 'Month', DIMIX( 'Month', sMonth ) ) @= ' ' );
    LogOutput( 'WARN', 'Ligne ignorée - Month inconnu: [' | sMonth | ']' (valeurs valides: Jan, Feb, Mar...Dec) );
    nLignesIgnorees = nLignesIgnorees + 1;
    ItemSkip();
ENDIF;

#---- 8. Validation : Year (membres réels : Y1, Y2, Y3, Y4) ----
IF( DIMNM( 'Year', DIMIX( 'Year', sYear ) ) @= ' ' );
    LogOutput( 'WARN', 'Ligne ignorée - Year inconnu: [' | sYear | ']' (valeurs valides: Y1, Y2, Y3, Y4) );
    nLignesIgnorees = nLignesIgnorees + 1;
    ItemSkip();
ENDIF;

#---- 9. Validation : valeurs numériques cohérentes ----
IF( nUnitsSold < 0 % nUnitPrice < 0 % nUnitCost < 0 );
    LogOutput( 'WARN', 'Ligne ignorée - valeur négative détectée: Org=' | sOrg | ' Prod=' | sProduct | ' ' );
    nLignesIgnorees = nLignesIgnorees + 1;
    ItemSkip();
ENDIF;

#---- 10. Écriture dans le cube Revenue ----
#    Ordre des dimensions (confirmé live) :
#    organization, Channel, product, Month, Year, Version, Revenue

CellPutN(
    nUnitsSold,
    'Revenue',
    sOrg, sChannel, sProduct, sMonth, sYear, sVersion,
    'Units Sold'
);

CellPutN(
    nUnitPrice,
    'Revenue',
    sOrg, sChannel, sProduct, sMonth, sYear, sVersion,
    'Unit Price'
);

```

```
CellPutN(
    nUnitCost,
    'Revenue',
    sOrg, sChannel, sProduct, sMonth, sYear, sVersion,
    'Unit Cost'
);

nLignesChargees = nLignesChargees + 1;
```

Epilog

EPILOG

Exécuté **une seule fois après** la lecture complète du fichier. Log du bilan, erreur finale si aucune ligne chargée.

```
#####
# Load_Revenue_FromCSV - EPILOG
# Objectif : log du bilan, erreur si rien chargé
#####

#---- 1. Log du bilan de chargement ----
LogOutput( 'INFO', '=== Load_Revenue_FromCSV - BILAN ===' );
LogOutput( 'INFO', 'Fichier      : ' | pFilePath );
LogOutput( 'INFO', 'Version      : ' | pVersion );
LogOutput( 'INFO', 'Lignes chargées : ' | NumberToString( nLignesChargees ) );
LogOutput( 'INFO', 'Lignes ignorées : ' | NumberToString( nLignesIgnorees ) );
LogOutput( 'INFO', 'Lignes en erreur: ' | NumberToString( nLignesErreur ) );

#---- 2. Erreur critique : aucune ligne chargée ----
IF( nLignesChargees = 0 );
    LogOutput( 'ERROR', 'Load_Revenue_FromCSV: AUCUNE ligne chargée - vérifier le fichier et les paramètres' );
    ProcessError();
ENDIF;

#---- 3. Avertissement si taux d'erreur > 10% ----
IF( nLignesChargees + nLignesIgnorees > 0 );
    nTauxErreur = nLignesIgnorees \ ( nLignesChargees + nLignesIgnorees ) * 100;
    IF( nTauxErreur > 10 );
        LogOutput( 'WARN', 'Taux d ignorées élevé: ' | NumberToString( ROUND(nTauxErreur,1) ) | '% - vérifier' );
    ENDIF;
ENDIF;

LogOutput( 'INFO', 'Load_Revenue_FromCSV: Terminé avec succès.' );
```

Récapitulatif technique

MEMBRES RÉELS VALIDÉS LIVE

DIMENSION	MEMBRES FEUILLES ÉCRITS	MEMBRES EXCLUS
organization	101-103, 201-202, 301-302, 401-403	Total Company, régions (consolidations)
Channel	10, 20, 30	Channel Total
product	21001/21002/21003, 22001/22002, 23001, 31001/31002/31004, 32001/32002/32003, 33001/33002/33003, 41010/41011/41012...	Product Total, consolidations (20000, 30000...)
Month	Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec	Year, Q1-Q4, YTD, FcstMonth, RowFormat, Description
Year	Y1, Y2, Y3, Y4	Y2-Y1 \$, Y2-Y1 % (membres variance)

Version	Version 1/2, Actual, Forecast, Target, Prior Year Actual, Predictive, Predictivev2	Variance, Rate Variance, Ccy Exchange Variance... (bloqués en Prolog)
Revenue (mesures)	Units Sold, Unit Price, Unit Cost	Gross Revenue, Cost of Sales, Gross Margin (calculés par règles)

POINTS DE CONCEPTION

DÉCISION	JUSTIFICATION
pVersion fait foi, pas V6	Un CSV peut contenir plusieurs versions mélangées — le paramètre garantit l'atomicité du chargement par version
On n'écrit que Units Sold, Unit Price, Unit Cost	Gross Revenue, Cost of Sales, Gross Margin sont calculés par règles TM1 — les écraser par TI casserait les règles
Validation DIMIX sur chaque dimension	Évite les erreurs silencieuses — TM1 crée de nouveaux membres si CellPutN est appelé avec un membre inexistant quand les règles de sécurité le permettent
Versions calculées bloquées en Prolog	8 versions calculées identifiées live (Variance, Rate Variance, Ccy Exchange Variance...) — écrire dedans produirait des données incohérentes
ItemSkip sur lignes invalides (pas ProcessBreak)	Une ligne mal formée ne doit pas interrompre le chargement complet — sauf erreur critique détectée en Prolog
Compteur taux d'erreur > 10%	Alerte sans arrêt — permet de détecter un fichier mal formaté sans bloquer le chargement